

Organização e Recuperação de Dados

1º Trabalho Prático

O 1º trabalho prático da disciplina **Organização e Recuperação de Dados** consiste na construção de um programa a ser desenvolvido em conformidade com as especificações abaixo. O programa deverá ser escrito na linguagem Python e poderá ser feito em equipes de até 2 alunos.

A equipe deverá enviar o código fonte pelo *Google Classroom* em um arquivo nomeado com os nomes dos integrantes da equipe. **O trabalho deverá ser apresentado pela equipe em data e horário agendado pela professora. A entrega do código é um pré-requisito para a avaliação, mas a nota dependerá do desempenho de cada integrante da equipe na apresentação. O uso deliberado de IA generativa poderá acarretar diminuição na nota.**

Especificação

O objetivo deste trabalho é indexar um arquivo de registros sobre jogos usando **índices lineares**. Os dados dos jogos estarão armazenados no arquivo *games.dat* em registros de tamanho variável que iniciam um campo de tamanho fixo com 2 bytes, em formato similar ao utilizado nas aulas práticas. Cada jogo possui os seguintes campos, nesta ordem:

- Identificador único do jogo (chave primária);
- Nome;
- Ano de lançamento;
- Gênero (chave secundária 1);
- Publicadora (chave secundária 2); e
- Plataforma principal.

O programa deverá oferecer as seguintes funcionalidades:

- Construção dos índices;
- Execução de operações de busca, inserção e remoção; e
- Compactação do arquivo de registros.

Construção dos índices

A funcionalidade de construção dos índices será acessada pela linha de comando, da seguinte forma:

```
$ python programa.py -b
```

sendo *programa.py* o nome do arquivo com o seu código e sendo *-b* a flag que sinaliza o modo de criação de índices. Sempre que ativada, essa funcionalidade construirá, a partir do arquivo *games.dat*, três índices lineares: um **índice primário**, um **índice secundário por gênero** (chave secundária 1) e um **índice secundário por publicadora** (chave secundária 2). Caso os arquivos de índice existam, eles deverão ser sobrescritos. Note que o formato do arquivo de registros será sempre igual, porém o número de jogos contidos no arquivo pode variar. Os índices secundários deverão ser implementados por **ligação tardia (*late binding*) usando uma lista invertida**, de acordo com as estruturas de dados vistas em aula. Ao final da execução, quatro arquivos deverão ter sido criados: *primario.ind*, *genero.ind*, *publicadora.ind* e *listaInvertida.lst*.

Execução de operações de busca, inserção e remoção

As operações a serem realizadas em determinada execução (*-e*) serão especificadas em um arquivo de operações, o qual será passado ao programa por parâmetro. Dessa forma, **o programa não possuirá interface com o usuário** e executará as operações na sequência em que estiverem especificadas no arquivo de operações.

A execução do arquivo de operações será acionada pela linha de comando, no seguinte formato:

```
$ python programa.py -e arquivo_operacoes
```

sendo `-e` a flag que sinaliza o modo de execução e `arquivo_operacoes` o nome do arquivo que contém as operações a serem executadas. Para simplificar o processamento do arquivo de operações, considere que ele sempre será fornecido no formato correto (i.e., o seu programa não precisa verificar a integridade desse arquivo).

Para que as operações sejam executadas, **os arquivos de índice e o arquivo de deverão existir**. Caso não existam, o programa deverá emitir uma mensagem de erro e terminar. **Para a execução das operações, os índices devem ser carregados para a memória**. Assim, durante a execução das operações, os índices serão mantidos e acessados em memória, enquanto **o arquivo de registros deverá ser mantido e acessado no dispositivo de memória secundária**. Os arquivos de índice serão atualizados no dispositivo de armazenamento sempre que uma execução for encerrada.

Formato do arquivo de operações

O arquivo de operações possuirá uma operação por linha, codificada com o identificador da operação seguido pelo seu argumento. Os identificadores de operações possíveis são: `bp` = *busca no índice primário*, `bs1` = *busca no índice secundário por gênero*, `bs2` = *busca no índice secundário por publicadora*, `i` = *inserção* ou `r` = *remoção*. A seguir é exemplificado o formato de um arquivo de operações.

```
bp 459
bs1 Hack and Slash
i 539|Minecraft|2011|Sandbox|Microsoft Studios|PC|
r 121
r 754
bs2 CD Projekt RED
```

O arquivo acima representa a execução consecutiva das seguintes operações:

- Busca no índice primário pelo registro de ID 205
- Busca no índice secundário por gênero pelos registros do gênero Hack and Slash
- Inserção do jogo 539 (Minecraft)
- Remoção do registro de chave 121
- Remoção do registro de chave 754
- Busca no índice secundário por publicadora pelos registros da publicadora CD Projekt

Com base no arquivo de operações mostrado acima, o programa deverá apresentar a seguinte saída:

```
Busca pelo registro de ID "459"
459|Fortnite|2017|Sandbox|Epic Games|PC|

Busca por registros de gênero "Hack and Slash" (5 registros)
100|God of War|2005|Hack and Slash|Sony|PlayStation 2|
127|Devil May Cry 3: Dante's Awakening|2005|Hack and Slash|Capcom|PlayStation 2|
238|NieR: Automata|2017|Hack and Slash|Square Enix|PlayStation 4|
728|Bayonetta 2|2014|Hack and Slash|Nintendo|Nintendo Switch|
964|Diablo II|2000|Hack and Slash|Blizzard|PC|

Inserção do registro de chave "539" (48 bytes)

Remoção do registro de chave "121" (offset = 936)

Remoção do registro de chave "754"
Registro não encontrado!

Busca por registros da publicadora "CD Projekt RED" (3 registros)
163|Cyberpunk 2077|2020|Action RPG|CD Projekt RED|PC|
875|The Witcher III: Wild Hunt|2015|Action RPG|CD Projekt RED|PC|
894|The Witcher 2: Assassins of Kings|2011|Action RPG|CD Projekt RED|PC|
```

Busca

As buscas devem ocorrer nos índices em memória. O acesso aos registros que correspondem ao resultado da busca deve ser feito de forma direta, por meio dos seus byte-offsets.

Inserção

IDs duplicados não são permitidos, logo, antes de realizar uma inserção, o programa deverá se certificar de que o ID do jogo a ser inserido não existe. Caso exista um jogo com o mesmo ID, deverá ser dada uma mensagem de ID duplicado e o registro será descartado. Caso contrário, a inserção do registro do jogo ocorrerá no final do arquivo *games.dat* e os índices serão atualizados.

Remoção

A remoção de registros no arquivo *games.dat* será lógica e o espaço do registro removido será fisicamente liberado apenas com a compactação do arquivo. Os índices serão atualizados. Note que no caso de uma remoção, apenas o índice primário e a lista invertida precisarão ser reorganizados.

Compactação do arquivo de registros

A funcionalidade de compactação do arquivo *games.dat* também será acessada via linha de comando, no seguinte formato:

```
$ python programa.py -c
```

sendo `-c` a flag que sinaliza o modo de compactação. Sempre que ativada, essa função deverá liberar fisicamente os espaços ocupados por registros removidos, resultando em um novo arquivo *games.dat* sem fragmentação. Note que quando o arquivo for compactado, o arquivo de índice primário deverá ser reconstruído e atualizado, caso exista.

BOM TRABALHO!